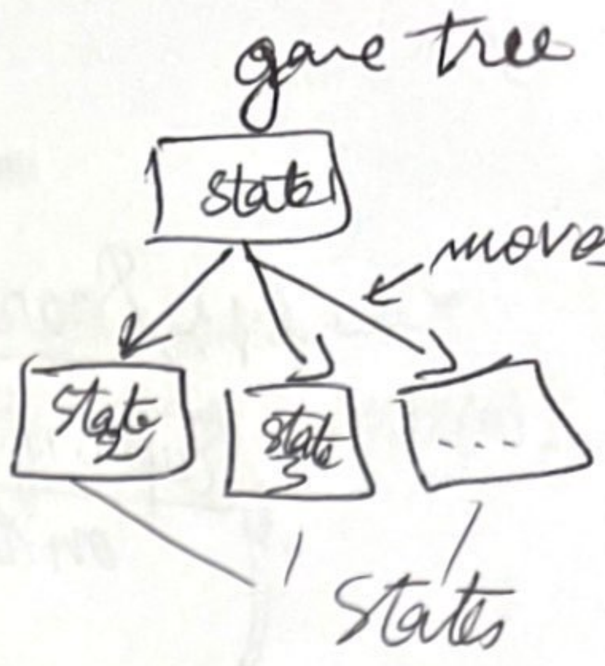
Games Adversarial Search



[✱ Environments are Competitive
✱ Agents' goals are in Conflict ✱ ✱

Games in AI are deterministic as (theorists said) game
- Turn taking
- two player
- Zero-sum games
- Perfect information.

✱ At the end the utility values of the game are always Equal and Opposite
one player wins and the other loses.

✱✱✱ Inefficiency is penalized severly!

① Consider games of 2 players: MAX and Min
States

↳ at the end → loser: gets pointslities
→ Winner: gets points

1

✱ Defining a Game Formally:

- S_0: the initial state
- Player(s): Which player has the move in a state
- Action(s): legal moves in a state
- Transition model: result of a move:
  Result(s,a)
- Terminal-test(s): true when the game is over and false otherwise.
  ⟶ if it's true then s is a terminal state

UTILITY(s,p):
= utility + payoff funct + objective.
→ defines the final numeric value for a game that ends in terminal state s for player p

Zero sum: $\Sigma 0$ game: the total payoff to all players is the same for every instance of the game (constant sum) like 1+0 / 0+1 / ½+½

Scanned with CamScanner

Game tree: is smaller than search tree.

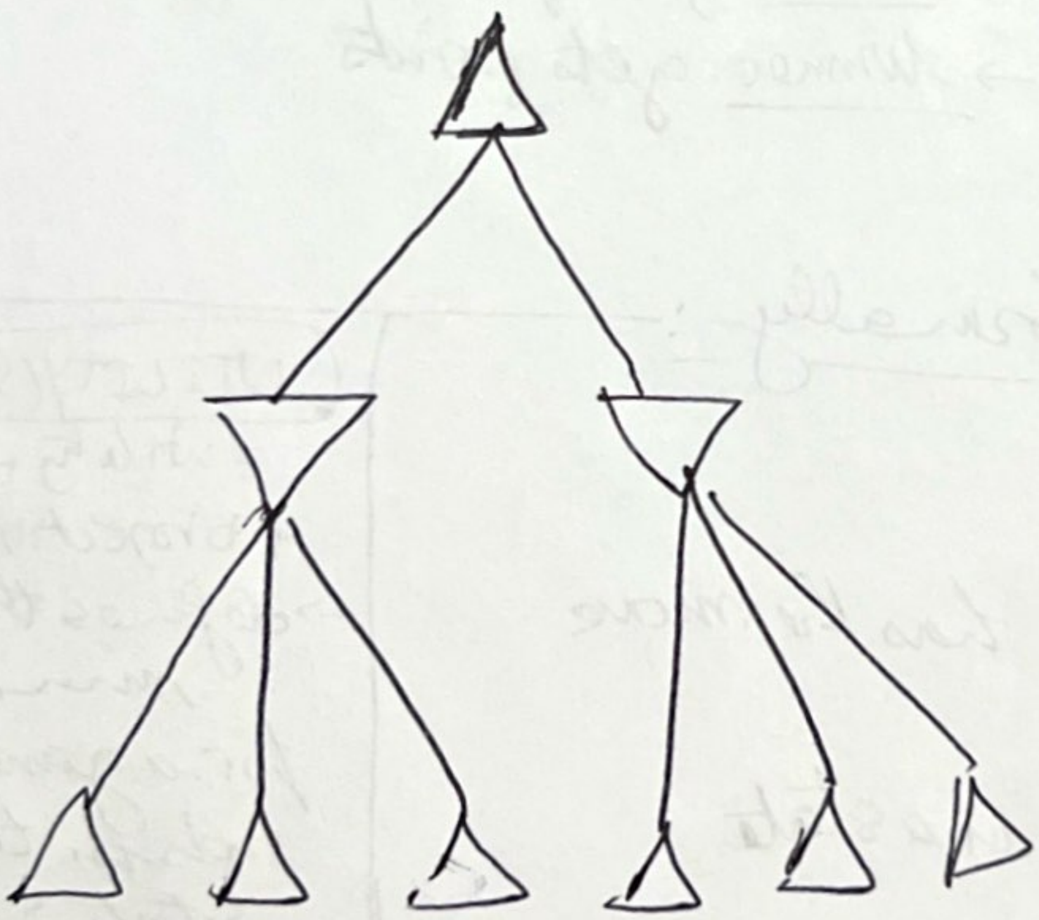
*** Search tree: Examines enough nodes to allow a player to determine what move to take.

(Superimposed *** on the full game tree)

2) Optimal Solutions Decisions in Games:

In adv. search MIN has a role, so MAX must find a contingent strategy which specify their move in initial state → then in the states resulting from every possible action by MIN!

* Optimal Solution Strategy: leads to outcomes at least as good as any other strategy when one is playing an infallible opponent.



2

one ply

the tree is one more deep ONLY!

Scanned with CamScanner

③ MiniMax Strategy:-

- the optimal strategy: determined from the minimax value of each node (MINIMAX(n))

- Definition: Minimax value of a node is the utility(for MAX) being in the corresponding state (WHERE both players play optimally!) from that node to the end!

$$MINIMAX(s) = \begin{cases} UTILITY(s) & \text{if TERMINAL-Test}(s) \\ \max_{a \in Actions(s)} MINIMAX(Result(s,a)) & \text{if } p = Max \\ \min_{a \in Action(s)} MINIMAX(Result(s,a)) & \text{if } p = Min \end{cases}$$

MINIMAX Decision is at the "ROOT"

→ Min not playing optimally ⇒ MAX will do even better.

the Algorithm in the slides performs a complete Depth-first exploration of the game tree

- time complexity: $O(b^m)$
  - b: branching factor
  - m: max depth of the tree
- space complexity: ① $O(bm)$ (actions generated ALL at once.)
  ② $O(m)$ if actions are generated one at a time!

Scanned with CamScanner

Extending the Idea to more than 2 players!

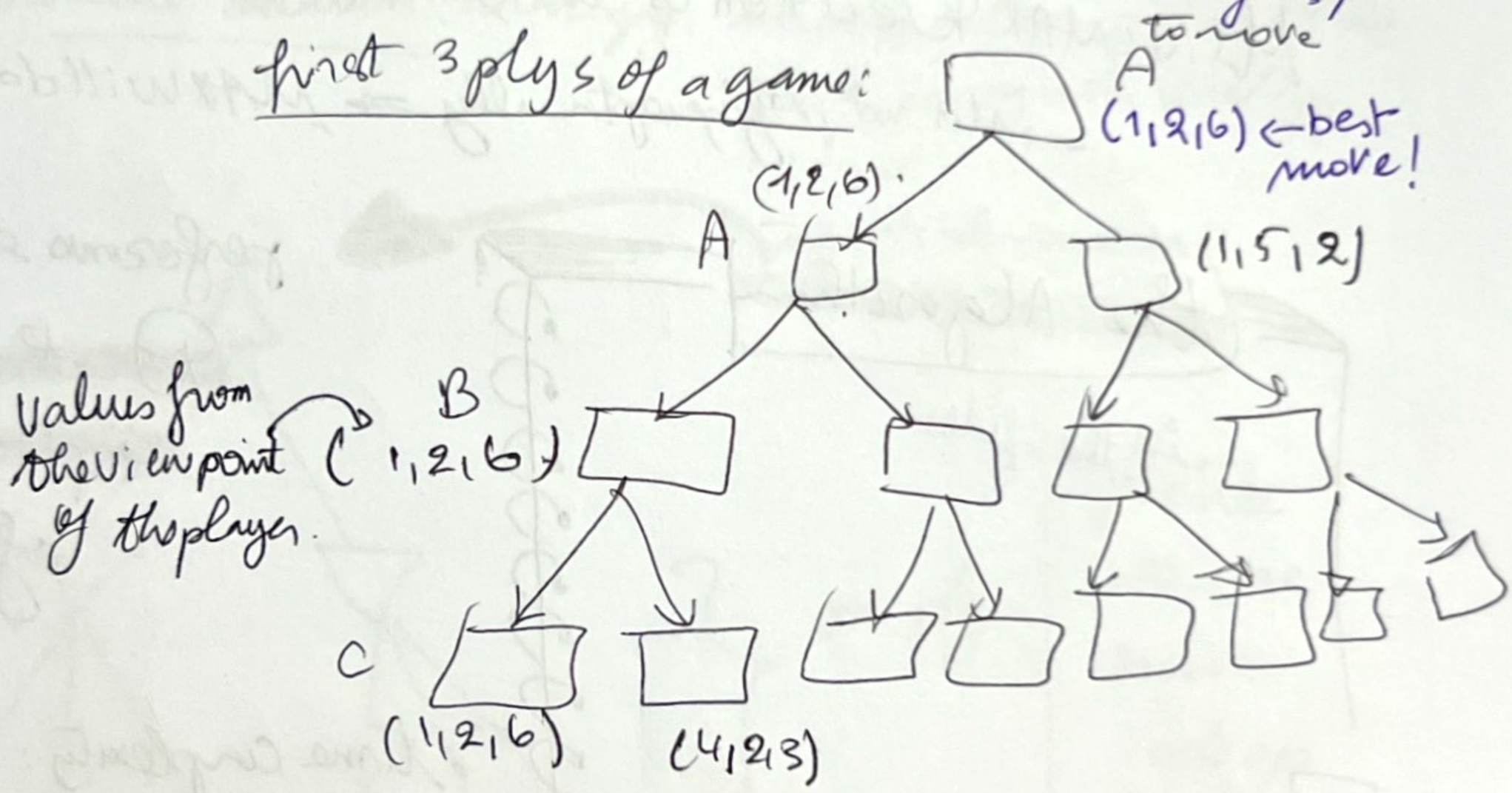
- Each node value will be replaced by a vector of values $\begin{matrix} A & B & C \\ V_A & V_B & V_C \end{matrix}$ all associated with each node.

- In terminal: it gives utility of the state from each player viewpoint

Backing up the values:

- the value of a node n (the vector) is always the utility vector of the successor state with highest value for the player choosing at n! (take the whole vector of c!)

first 3 plys of a game:



4

☆ Alliances: can be formal or informal.

- They change as the game proceeds.
- Can be natural consequence of optimal strategy for each player in multiplayer games.

A and B are weak and C is in a stronger position

A and B ⟶ attack ☆ C.

⟶ C will be weakened
⟶ alliance will lose its value
A or B could change the approach.

Scanned with CamScanner

✧ α-β Pruning: two parameters that describe bounds on the backed-up values

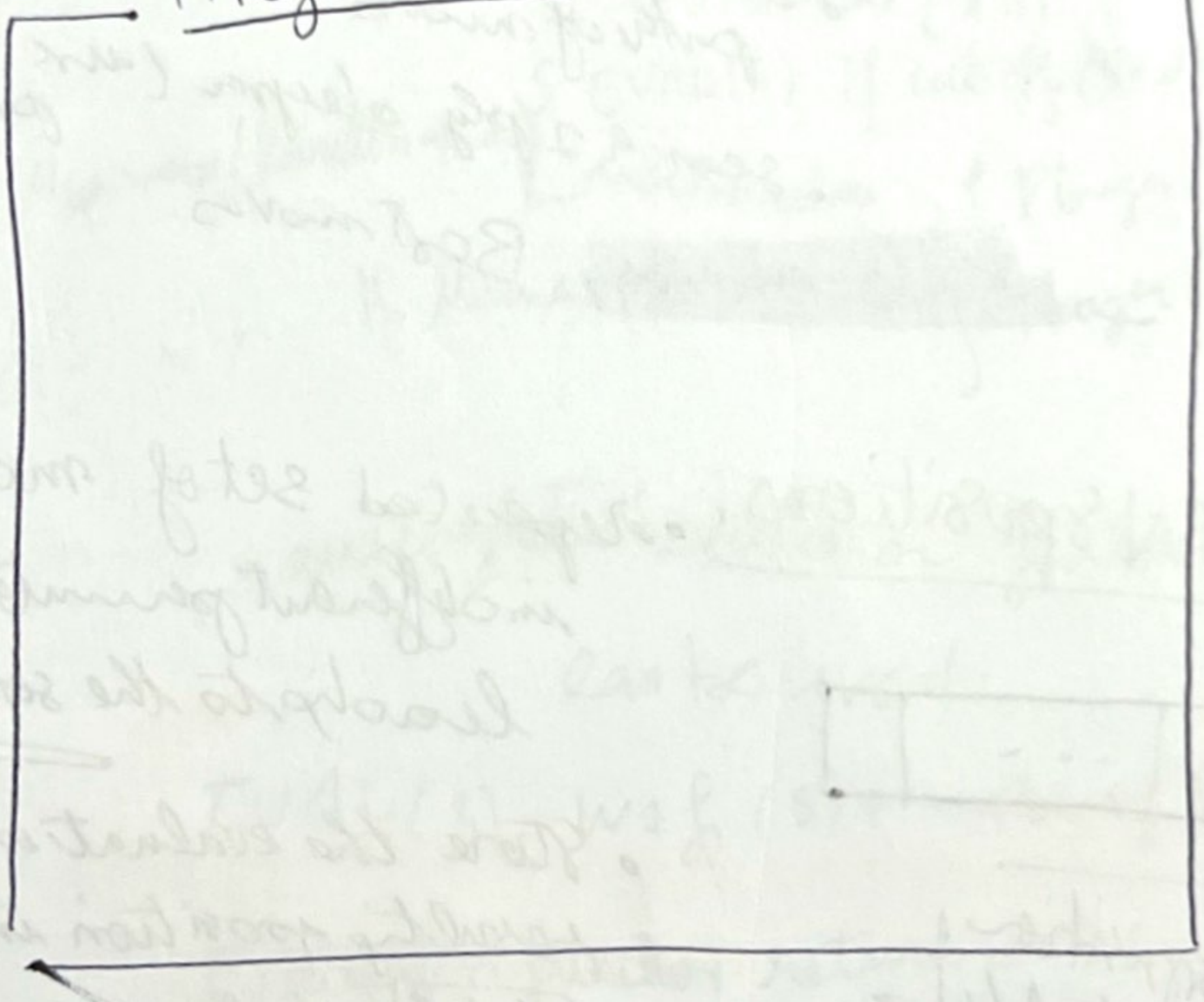
✧ α: (highest-value) "the best" choice so far found along the path for MAX

✧ β: (lowest-value) "the best" choice " " " for MIN

→ Updates the values α/β and terminates the recursive call) as soon as: ⚠ the value of the current node is known to be worse than the current α (resp. β) value for MAX (resp MIN).

"The example until slide 58"

+ Algorithm: Double recursive



Move Ordering

> Effectiveness is highly dependent on the order in which states are examined

▷ We would be able to prune more trees if the order was different.

▷ Suggestion: EXAMINE first the successors that are likely to be best.

if it is done: $O(b^{m/2})$ nodes must be examined to pick the best move instead of $O(b^m)$ for minimax.

branching factor will be $\sqrt{B}$

Scanned with CamScanner

• Results: → α-β can solve a tree roughly twice as deep as minimax in the same amount of time

▷ $O(b^{3m/4})$ must be roughly examined, if successors are examined at random.

☆ Simple ordering functions can be used to make the results better like: $O(b^{m/2})$ best-case result.

☆ Add Dynamic move ordering schemes

→ try best moves that were found to be best in part

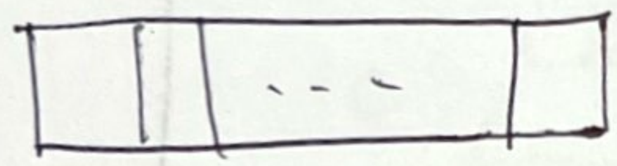
☆ From the ways to get information from the current move with IDS

- Search 1 ply deep and record the best path of moves
- search 1 ply deeper (use recorded path)

"Repeated State occur frequently" → Killer moves, Best moves.

→ TRANSpositions:

- repeated set of moves in different permutations leading to the same result.
- Store the evaluation of the resulting position in a hash table (the first time it is encountered)



transposition table

↳ using it can have a Dramatic effect such as doubling the reachable search depth in chess

problem

(it is not practical to keep all of the nodes in the transposition table (various strategies have been used to choose which nodes

Scanned with CamScanner

Imperfect Real-time decisions:

Claude Shanon > Cut off the search earlier, apply a heuristic evaluation function to states in the search to turn nonterminal nodes into terminal leaves.

☆ How it alters minimax or dyplaleta:

① Replace the utility function by a heuristic evaluation function EVAL (estimates the position's utility

② Replace terminal test that decides When to apply EVAL

↳ Result:

H_MiniMax(s,d)
EVAL(s) if cuttoff(s,d).
H_Minimax(Result(s,a),d+1) ; if Player(s)=Max
maxa∈actions
H_Minimax(Result(s,a),d+1); if Player(s) = MIN.

slide 67

a weighted evaluation function

can be used:

$$EVAL(s) = w_1 f_1(s) + \cdots + w_n f_n(s) = \sum_{i=1}^{n} w_i f_i(s)$$

Evaluation function: returns an estimate of the expected utility of the game from a given position.

- best to make guess on the final outcome.
  + Consider various features of the state

(needs human expertise)

Sometimes they use nonlinear combinations of features.

- features need strong assumptions, and the contribution of each feature is independent of the values of other features

← (some values will change at the end of the game)

Scanned with CamScanner

Slide 71 -

When cutting off search: More robust approach is to apply iterative deepening. When time runs out, the program returns the move selected by the deepest completed search.

→ problem: Can prevent to see interesting potential developments that are one more ply ahead.

Quiescence search.

→ Apply the eval function for Quiescent positions

Quiescent positions ⊂ Non quiescent

Non quiescent → Expand further

Quiescent positions: unlikely to change in the near future. (but the ones with favourable captures for a heuristic that counts material)

☆ Horizon Effect: when the opponent's move causes a serious damage and it is ultimately unavoidable. But it can be temporarily avoided by delaying tactics.

↙ more difficult to eliminate.

→ Solution strategy.

■ Singular extensions: move that is clearly "better" than all other moves in a given position. (memorized during the tree search until depth limit)

↙ make the tree deeper.

Scanned with CamScanner

Forward Pruning: some moves pruned, like when humans consider few moves from each position.

→ approach: **beam search**: on each ply

> Consider a beam of *n* best moves according to the evaluation function instead of all possible moves.

But risk!
of pruning better/best moves.

Search Vs Lookup:

✧✧✧ **table lookup**: Many games use it, rather than search for the opening and ends of games which rely on Expertise of humans
→ gathering statistics from a database of previously played games to see which opening sequences most often lead to a win

* Usually, after 10 moves, we end up in rarely seen positions → switch to search in that case.

LOOKUP - STRATEGY

We have the relation

bigger chance to do lookup ↑

fewer possible positions ↓

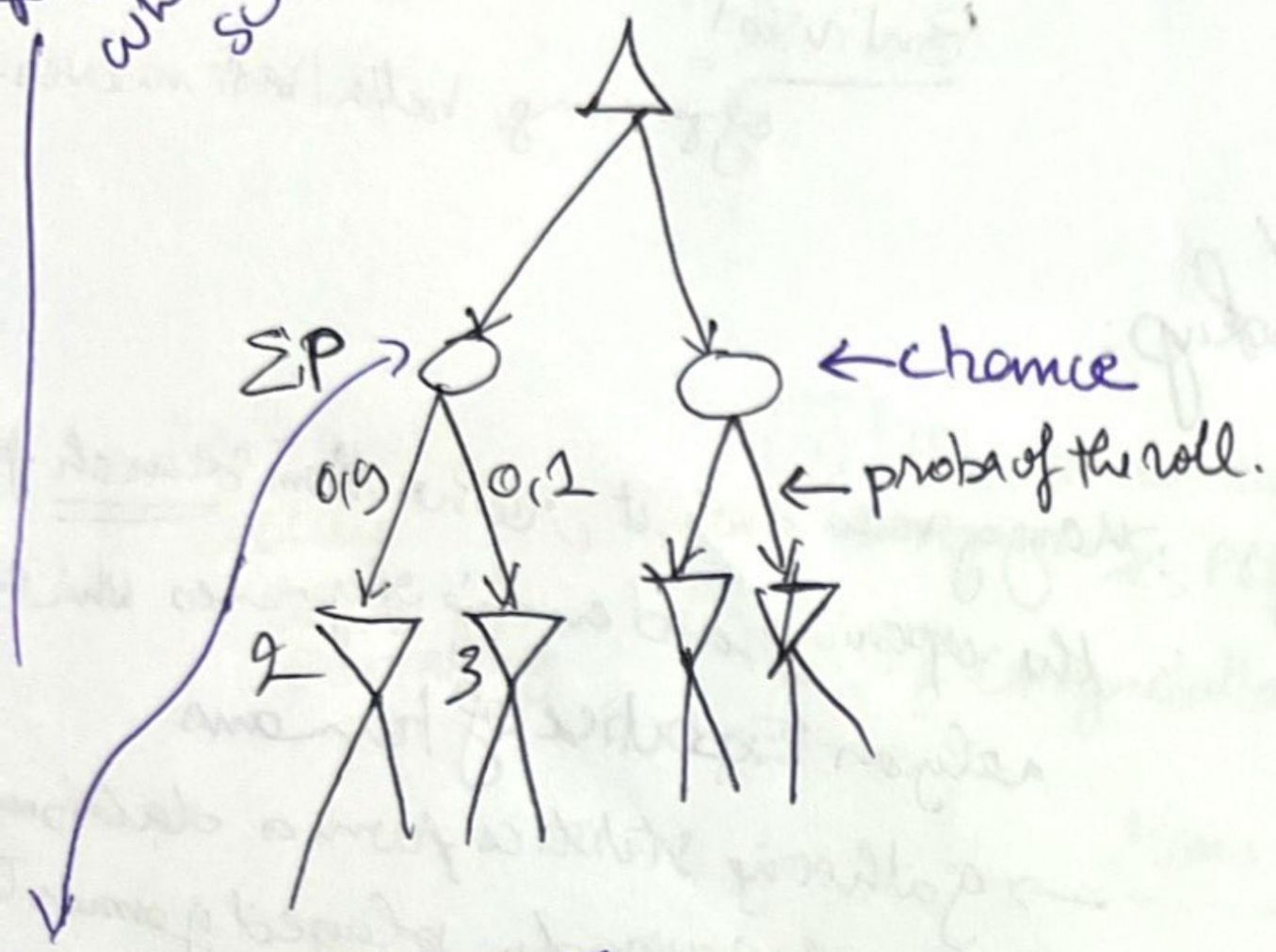
A computer can solve the endgame by producing a policy, which is a mapping from every possible state to the best move in the state.

Scanned with CamScanner

Stochastic Games:

- mirror unpredictability
- they include a random element. EX: throwing a dice.
- Includes chance nodes in addition to MAX and MIN.

Different behaviors when we change the scale of the evaluations



Can only:

1) Calculate the expected-value of a position: the average over all possible outcomes of the chance nodes

$$\sum_r P(r)$$

$2 \times 0,9 + 0,1 \times 3 = 2,1$ Average overall possible outcomes.

EXPECTIMINIMAX It will take $O(b^m n^m)$

b: branching factor. m: max depth. n: the number of distinct rolls.

Alpha β pruning can be applied to game trees with chance nodes.

→ find an upper bound for the chance node if found.

EXPECTIMINIMAX(S)

$$= \begin{cases} \text{UTILITY}(S) & \text{if Terminal-state}(S) \\ \max_a \text{EXPECTIMINIMAX}(\text{Result}(S,a)) & \text{if player}(S) = \text{MAX} \\ \min_a \text{EXPECTIMINIMAX}(\text{Result}(S,a)) & \text{if player}(S) = \text{MIN} \\ \sum_r P(r)\ \text{EXPECTIMINIMAX}(\text{Result}(S,a)) & \text{if player}(S) = \text{Chance.} \end{cases}$$

Scanned with CamScanner

Partially observable Games: Card games:

Stochastic - partial observability

→ Missing info can be generated at random.

Ideas to solve:
- solve each possible deal of the invisible cards. as if it was a fully observable game
- choose the move that has the best outcome averaged over all the deals.

① the node we evaluate is:
argmax: $\sum_s P(s)$ MINIMAX(Result(s,a))

② If computationaly feasible:
run MINIMAX. otherwise:
H_MINIMAX.

each deal s occurs with probability P(s).

Problem: Number of possible deals is LARGE

→ instead of adding up all the deals: take random sample of N deals where the probability of deal s appears in the sample is proportional to P(s).

$$\text{argmax} \frac{1}{N} \sum_{i=1}^{N} \text{MINIMAX}(\text{Result}(s,a)).$$

Scanned with CamScanner